COURSE PROJECT REPORT

Design and Implementation of a Database Management System for Online Course Learning Progress Management System

Database Management Systems (DBMS) Laboratory Course Project

Student Name	Roll Number	Section
Siddhant Agarwal	25WU0102262	AIML Whales
Manthena Naga Sri Rohan	25WU0102151	AIML Panthers

Course	Database Management Systems (DBMS) Laboratory
Program	B.Tech in CSE (Artificial Intelligence & Machine Learning), 2nd Year
Department	Computer Science & Engineering (AI & ML)
Faculty Name	Course Faculty / Evaluator, Department of CSE (AI & ML)
Academic Year	2026-2027
Database Engine	MySQL Server 9.7 (InnoDB)
Architecture	3-Tier: React/Vite frontend, Node.js/Express REST API, MySQL
GitHub	github.com/asiddhant2405/DBMS-Course-Project

Table of Contents

1	Cover Page
2	Abstract
3	Introduction and Problem Statement
4	Objectives and Scope
5	Software and Hardware Requirements
6	ER Diagram
7	Relational Schema and Normalization
8	Data Dictionary (Table Structure, Data Types, Constraints)
9	SQL Commands Used (DDL, DML) with Sample Outputs
10	Queries with Outputs, Including the Query Given During Presentation-II
11	UI Design and Screenshots
12	Implementation Details (Technology Stack, DB Connectivity, Key Code Snippets)
13	Testing (Test Cases and Results)
14	Conclusion and Future Enhancements
15	References
16	Appendix: GitHub Repository Link

2. Abstract

In modern e-learning platforms, tracking academic progress across multi-module curricula, varied assessment types and diverse student cohorts is a significant data governance challenge. Naive systems suffer from data redundancy, orphaned progress records, grade inconsistencies and a lack of real-time aggregate visibility.

This project presents the **Online Course Learning Progress Management System**, a relational database application built for the B.Tech 2nd-year DBMS laboratory evaluation. The system models an 8-entity schema: STUDENT, INSTRUCTOR, COURSE, MODULE, ASSESSMENT, ENROLLMENT, PROGRESS and RESULT. The schema is normalized to Boyce-Codd Normal Form (BCNF) and enforces referential integrity through RESTRICT foreign keys, CHECK constraints and composite UNIQUE keys.

The application connects to a live MySQL 9.7 server through a Node.js connection pool (mysql2/promise) and provides a React + Vite + TypeScript dashboard. Key capabilities are full CRUD on all eight entities, live KPI aggregation, an interactive SQL Query Lab that runs multi-table JOIN queries with execution timing, and a guided Faculty Evaluation flow demonstrating INSERT, SELECT, DELETE and foreign-key protection against the real database.

3. Introduction and Problem Statement

3.1 Background and Motivation

E-learning systems generate continuous educational data: students register for courses, work through ordered modules, attempt quizzes and exams, and accumulate completion metrics. Managing this in unnormalized structures leads to problems such as:

- A student recorded as completing a module of a course they never enrolled in.
- Course deletion silently wiping historical grades.
- Multi-valued attributes (for example, lists of courses in one column) that hinder reporting.

3.2 Problem Statement

To design, implement, normalize and demonstrate a relational database system that:

- Prevents duplicate registrations and invalid scores through declarative constraints.
- Maintains parent-child dependencies across COURSE, MODULE and ASSESSMENT.
- Tracks module completion (0 to 100 percent) and assessment scores (0 to Max Marks) accurately.
- Gives faculty and administrators an interface to run relational queries and monitor progress in real time.

4. Objectives and Scope

4.1 Core Objectives

- **Relational integrity:** eight connected tables with primary, foreign, unique and check constraints.
- **BCNF normalization:** every determinant in every non-trivial functional dependency is a candidate key.
- **No simulation:** all frontend screens read from a real MySQL instance; no mock APIs, localStorage or in-memory arrays.
- **DBMS concepts:** joins, aggregation with GROUP BY, outer joins and transactional error handling.
- **Deterministic seed data:** exactly 30 students (14 in Year 1) and Dr. Alan Turing teaching exactly 2 courses, to satisfy the Presentation-II benchmark queries.

4.2 System Scope

- **Student profiling:** ID, name, email, department (Computer Science, Information Technology, Electronics) and academic year.
- **Curriculum:** instructors assigned to courses, courses split into ordered modules, modules carrying assessments (Quiz, Assignment, Exam).
- **Registration and telemetry:** unique enrollments, per-module progress with timestamps, and graded results.

5. Software and Hardware Requirements

5.1 Hardware Requirements

Component	Requirement
Processor	64-bit multi-core CPU (Intel Core i5/i7 or AMD Ryzen 5/7, 2.0 GHz or higher)
RAM	Minimum 8 GB (16 GB recommended to run MySQL, Node.js and Vite together)
Storage	Minimum 500 MB free (SSD preferred)
Network	Local loopback (127.0.0.1 / localhost)

5.2 Software Requirements

Component	Requirement
Operating system	Windows 10/11 64-bit, Ubuntu 22.04 LTS or macOS
Database engine	MySQL Server 9.7 or 8.x (InnoDB, port 3306)
Backend runtime	Node.js v18.x or v20.x LTS, npm 9/10
Frontend	React 18, Vite 6, TypeScript 5, TailwindCSS 3.4, React Router 6, Recharts 2, Lucide Icons
Database tools	MySQL Workbench 8.0 CE or MySQL command-line client
Browser	Chrome, Edge or Firefox with modern ECMAScript support

6. ER Diagram

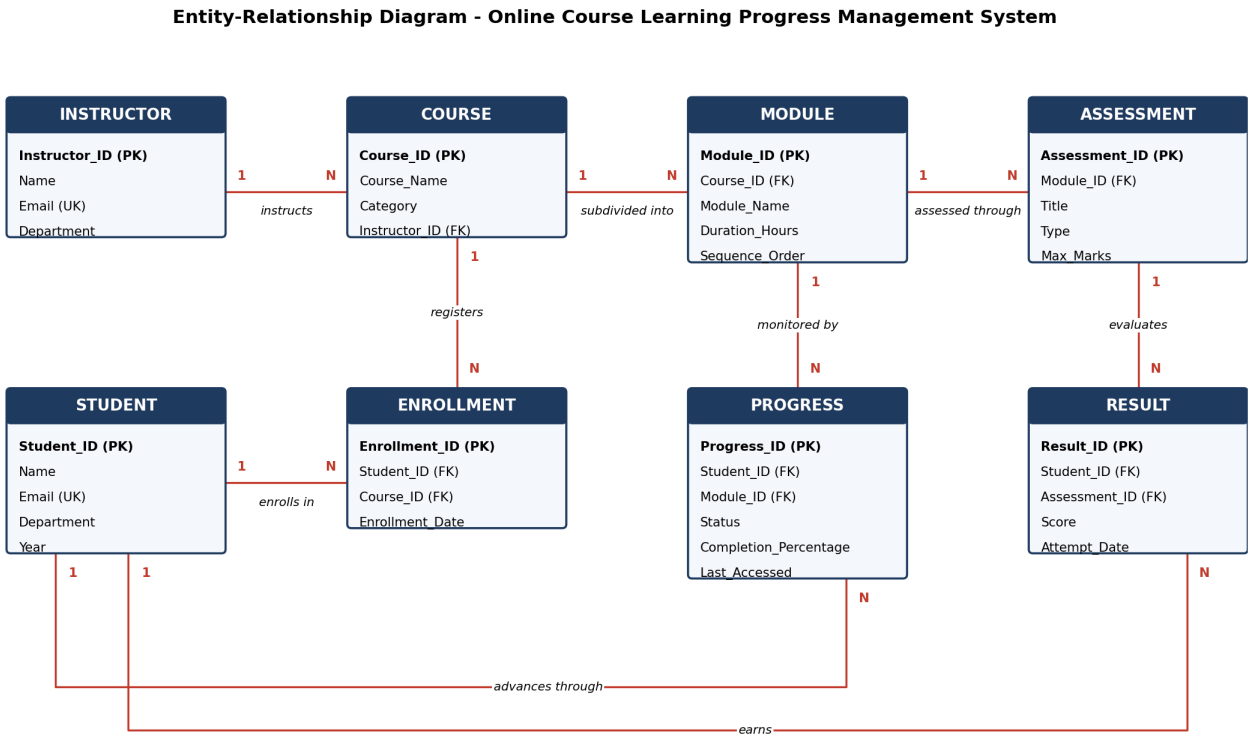


Figure 6.1: Entity-Relationship diagram (8 entities; PK = primary key, FK = foreign key, UK = unique key)

6.1 Entity Identification

Entity	Type	Description
STUDENT	Strong	Enrolled learners
INSTRUCTOR	Strong	Teaching faculty
COURSE	Dependent	Course offering owned by one instructor
MODULE	Dependent	Ordered chapter within a course
ASSESSMENT	Dependent	Quiz, assignment or exam inside a module
ENROLLMENT	Associative	M:N between STUDENT and COURSE
PROGRESS	Associative	M:N between STUDENT and MODULE
RESULT	Associative	M:N between STUDENT and ASSESSMENT

6.2 Cardinality and Participation

Relationship	Cardinality	Meaning
INSTRUCTOR - COURSE	1:N	An instructor teaches zero or more courses; each course has exactly one instructor.
COURSE - MODULE	1:N	A course has one or more modules; each module belongs to exactly one course.
MODULE - ASSESSMENT	1:N	A module has zero or more assessments; each assessment evaluates one module.
STUDENT - COURSE (ENROLLMENT)	M:N	Students enroll in many courses; courses register many students.
STUDENT - MODULE (PROGRESS)	M:N	A student tracks progress in many modules.
STUDENT - ASSESSMENT (RESULT)	M:N	A student attempts many assessments; each assessment has many results.

7. Relational Schema and Normalization

7.1 Relational Schema

Primary keys are marked (PK) and foreign keys (FK).

```
STUDENT    (Student_ID PK, Name, Email, Department, Year)
INSTRUCTOR (Instructor_ID PK, Name, Email, Department)
COURSE     (Course_ID PK, Course_Name, Category, Instructor_ID FK)
MODULE     (Module_ID PK, Course_ID FK, Module_Name, Duration_Hours, Sequence_Order)
ASSESSMENT (Assessment_ID PK, Module_ID FK, Title, Type, Max_Marks)
ENROLLMENT (Enrollment_ID PK, Student_ID FK, Course_ID FK, Enrollment_Date)
PROGRESS   (Progress_ID PK, Student_ID FK, Module_ID FK, Status,
            Completion_Percentage, Last_Accessed)
RESULT     (Result_ID PK, Student_ID FK, Assessment_ID FK, Score, Attempt_Date)
```

7.2 Functional Dependencies

Table	Functional dependencies
STUDENT	Student_ID -> Name, Email, Department, Year; Email -> Student_ID, Name, Department, Year
INSTRUCTOR	Instructor_ID -> Name, Email, Department; Email -> Instructor_ID, Name, Department
COURSE	Course_ID -> Course_Name, Category, Instructor_ID
MODULE	Module_ID -> Course_ID, Module_Name, Duration_Hours, Sequence_Order; (Course_ID, Sequence_Order) -> Module_ID
ASSESSMENT	Assessment_ID -> Module_ID, Title, Type, Max_Marks
ENROLLMENT	Enrollment_ID -> Student_ID, Course_ID, Enrollment_Date; (Student_ID, Course_ID) -> Enrollment_ID, Enrollment_Date
PROGRESS	Progress_ID -> all attributes; (Student_ID, Module_ID) -> Progress_ID, Status, Completion_Percentage, Last_Accessed
RESULT	Result_ID -> all attributes; (Student_ID, Assessment_ID) -> Result_ID, Score, Attempt_Date

7.3 Normalization Proof

First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic and there are no repeating groups. Every column holds a single scalar value, no comma-separated lists are stored, and multi-valued course registrations are modeled as separate ENROLLMENT rows. **The schema satisfies 1NF.**

Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and no non-prime attribute depends on a proper subset of a candidate key. Each table has a single-attribute surrogate primary key, so partial dependencies are impossible. For the composite candidate keys, Enrollment_Date depends on the full pair (Student_ID, Course_ID), Completion_Percentage on (Student_ID, Module_ID) and Score on (Student_ID, Assessment_ID). **The schema satisfies 2NF.**

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on a key. COURSE stores only Instructor_ID (not instructor name or department), MODULE stores only Course_ID, and ASSESSMENT stores only Module_ID. Descriptive details are obtained through JOINs, so no chain PK -> X -> Y among non-key attributes exists. **The schema satisfies 3NF.**

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial dependency X -> Y, X is a superkey. Checking all dependencies listed in 7.2: the determinants are Student_ID or Email (STUDENT), Instructor_ID or Email (INSTRUCTOR), Course_ID (COURSE), Module_ID or (Course_ID, Sequence_Order) (MODULE), Assessment_ID (ASSESSMENT), and the surrogate ID or the natural composite key in ENROLLMENT, PROGRESS and RESULT. Every determinant is a superkey. **The schema is in BCNF.**

8. Data Dictionary

8.1 STUDENT

Column	Data Type	Null	Key / Constraint	Description
Student_ID	INT	No	PK, AUTO_INCREMENT	Unique student identifier
Name	VARCHAR(100)	No	-	Full name
Email	VARCHAR(100)	No	UNIQUE	Institutional email
Department	VARCHAR(50)	No	-	Academic department
Year	INT	No	CHECK (Year IN 1,2,3,4)	Cohort year

8.2 INSTRUCTOR

Column	Data Type	Null	Key / Constraint	Description
Instructor_ID	INT	No	PK, AUTO_INCREMENT	Unique faculty identifier
Name	VARCHAR(100)	No	-	Name with title
Email	VARCHAR(100)	No	UNIQUE	Faculty email
Department	VARCHAR(50)	No	-	Teaching department

8.3 COURSE

Column	Data Type	Null	Key / Constraint	Description
Course_ID	INT	No	PK, AUTO_INCREMENT	Course identifier
Course_Name	VARCHAR(150)	No	-	Course title

Column	Data Type	Null	Key / Constraint	Description
Category	VARCHAR(50)	No	-	Subject category
Instructor_ID	INT	No	FK to INSTRUCTOR (RESTRICT)	Assigned instructor

8.4 MODULE

Column	Data Type	Null	Key / Constraint	Description
Module_ID	INT	No	PK, AUTO_INCREMENT	Module identifier
Course_ID	INT	No	FK to COURSE (RESTRICT)	Parent course
Module_Name	VARCHAR(150)	No	-	Module title
Duration_Hours	INT	No	CHECK (> 0)	Expected duration
Sequence_Order	INT	No	CHECK (> 0)	Order within course

8.5 ASSESSMENT

Column	Data Type	Null	Key / Constraint	Description
Assessment_ID	INT	No	PK, AUTO_INCREMENT	Assessment identifier
Module_ID	INT	No	FK to MODULE (RESTRICT)	Parent module
Title	VARCHAR(150)	No	-	Assessment title
Type	VARCHAR(50)	No	CHECK IN (Quiz, Assignment, Exam)	Assessment format
Max_Marks	INT	No	CHECK (> 0)	Maximum score

8.6 ENROLLMENT

Column	Data Type	Null	Key / Constraint	Description
Enrollment_ID	INT	No	PK, AUTO_INCREMENT	Enrollment identifier
Student_ID	INT	No	FK to STUDENT (RESTRICT)	Enrolled student
Course_ID	INT	No	FK to COURSE (RESTRICT)	Registered course
Enrollment_Date	DATE	No	DEFAULT CURRENT_DATE	Registration date

Table constraint: UNIQUE (Student_ID, Course_ID) prevents duplicate registrations.

8.7 PROGRESS

Column	Data Type	Null	Key / Constraint	Description
Progress_ID	INT	No	PK, AUTO_INCREMENT	Progress record id
Student_ID	INT	No	FK to STUDENT (RESTRICT)	Student
Module_ID	INT	No	FK to MODULE (RESTRICT)	Module
Status	VARCHAR(50)	No	CHECK IN (Not Started, In Progress, Completed)	Completion status
Completion_Percentage	INT	No	CHECK BETWEEN 0 AND 100	Percent complete
Last_Accessed	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP, ON UPDATE	Last activity

Table constraint: UNIQUE (Student_ID, Module_ID) gives one progress row per student per module.

8.8 RESULT

Column	Data Type	Null	Key / Constraint	Description
Result_ID	INT	No	PK, AUTO_INCREMENT	Result identifier
Student_ID	INT	No	FK to STUDENT (RESTRICT)	Student

Column	Data Type	Null	Key / Constraint	Description
Assessment_ID	INT	No	FK to ASSESSMENT (RESTRICT)	Assessment
Score	DECIMAL(5,2)	No	CHECK (>= 0)	Marks achieved
Attempt_Date	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Submission time

Table constraint: UNIQUE (Student_ID, Assessment_ID) prevents duplicate attempts.

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 DDL: Schema Definition (database/schema.sql)

```
CREATE DATABASE IF NOT EXISTS learningprogressdb;
USE learningprogressdb;

CREATE TABLE IF NOT EXISTS STUDENT (
    Student_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Department VARCHAR(50) NOT NULL,
    Year INT NOT NULL,
    CONSTRAINT chk_student_year CHECK (Year IN (1, 2, 3, 4))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS INSTRUCTOR (
    Instructor_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Department VARCHAR(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS COURSE (
    Course_ID INT AUTO_INCREMENT PRIMARY KEY,
    Course_Name VARCHAR(150) NOT NULL,
    Category VARCHAR(50) NOT NULL,
    Instructor_ID INT NOT NULL,
    CONSTRAINT fk_course_instructor FOREIGN KEY (Instructor_ID)
        REFERENCES INSTRUCTOR (Instructor_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS MODULE (
    Module_ID INT AUTO_INCREMENT PRIMARY KEY,
    Course_ID INT NOT NULL,
    Module_Name VARCHAR(150) NOT NULL,
    Duration_Hours INT NOT NULL,
    Sequence_Order INT NOT NULL,
    CONSTRAINT fk_module_course FOREIGN KEY (Course_ID)
        REFERENCES COURSE (Course_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_module_duration CHECK (Duration_Hours > 0),
    CONSTRAINT chk_module_sequence CHECK (Sequence_Order > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS ASSESSMENT (
    Assessment_ID INT AUTO_INCREMENT PRIMARY KEY,
    Module_ID INT NOT NULL,
    Title VARCHAR(150) NOT NULL,
    Type VARCHAR(50) NOT NULL,
    Max_Marks INT NOT NULL,
    CONSTRAINT fk_assessment_module FOREIGN KEY (Module_ID)
        REFERENCES MODULE (Module_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_assessment_type CHECK (Type IN ('Quiz','Assignment','Exam')),
    CONSTRAINT chk_assessment_marks CHECK (Max_Marks > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS ENROLLMENT (
    Enrollment_ID INT AUTO_INCREMENT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Course_ID INT NOT NULL,
    Enrollment_Date DATE NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT fk_enrollment_student FOREIGN KEY (Student_ID)
        REFERENCES STUDENT (Student_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_enrollment_course FOREIGN KEY (Course_ID)
        REFERENCES COURSE (Course_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_student_course UNIQUE (Student_ID, Course_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS PROGRESS (
    Progress_ID INT AUTO_INCREMENT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Module_ID INT NOT NULL,
    Status VARCHAR(50) NOT NULL,
    Completion_Percentage INT NOT NULL,
    Last_Accessed TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_progress_student FOREIGN KEY (Student_ID)
        REFERENCES STUDENT (Student_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_progress_module FOREIGN KEY (Module_ID)
        REFERENCES MODULE (Module_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_progress_status
        CHECK (Status IN ('Not Started','In Progress','Completed')),
    CONSTRAINT chk_progress_percentage
        CHECK (Completion_Percentage BETWEEN 0 AND 100),
    CONSTRAINT uq_student_module UNIQUE (Student_ID, Module_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS RESULT (
    Result_ID INT AUTO_INCREMENT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Assessment_ID INT NOT NULL,
    Score DECIMAL(5,2) NOT NULL,
    Attempt_Date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_result_student FOREIGN KEY (Student_ID)
        REFERENCES STUDENT (Student_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_result_assessment FOREIGN KEY (Assessment_ID)
        REFERENCES ASSESSMENT (Assessment_ID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_result_score CHECK (Score >= 0),
    CONSTRAINT uq_student_assessment UNIQUE (Student_ID, Assessment_ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9.2 DML: Seed Data (database/seed.sql)

The seed script populates the database deterministically to satisfy the course benchmarks:

Item	Seeded value
Students	30 total
Year distribution	Year 1: 14, Year 2: 10, Year 3: 6
Department distribution	Computer Science 13, Information Technology 10, Electronics 7
Instructors	3 (Dr. Alan Turing, Dr. Ada Lovelace, Dr. CV Raman)
Courses	4: Intro to Python (101, Programming) and Database Systems (102, Database), both by Dr. Alan Turing; Cloud & Web Architecture (Cloud Computing); Circuits & Systems (Hardware)
Enrollments / Progress / Results	30 enrollments, 60 progress records, 60 graded results

9.3 Sample DML Operations and Outputs

```
-- INSERT (Faculty demo, Step A)
INSERT INTO STUDENT (Name, Email, Department, Year)
VALUES ('Demonstration Student', 'demo.student@univ.edu', 'Computer Science', 1);
-- Output: Query OK, 1 row affected   (new Student_ID = 31)

-- SELECT (Step B)
SELECT * FROM STUDENT WHERE Student_ID = 31;
-- Output: 1 row returned

-- DELETE unreferenced record (Step C)
DELETE FROM STUDENT WHERE Student_ID = 31;
-- Output: Query OK, 1 row affected

-- DELETE referenced record (Step D) - blocked by ON DELETE RESTRICT
DELETE FROM STUDENT WHERE Student_ID = 1;
-- Output: ERROR 1451 (23000): Cannot delete or update a parent row:
--         a foreign key constraint fails

-- CHECK constraint violation
INSERT INTO PROGRESS (Student_ID, Module_ID, Status, Completion_Percentage)
VALUES (1, 1, 'In Progress', 150);
-- Output: ERROR 3819 (HY000): Check constraint 'chk_progress_percentage' is violated
```

Note: the demo student's email value above is illustrative; the actual value is generated by the application's demo step.

10. Queries with Outputs, Including the Query Given During Presentation-II

Presentation-II queries. Two benchmark queries were evaluated: (1) retrieve all first-year students (`WHERE Year = 1`), returning exactly 14 rows; and (2) an INSTRUCTOR-COURSE inner join showing that Dr. Alan Turing teaches exactly 2 courses. Both appear below as Query 1 and Query 2, followed by the wider analytical set.

Query 1: First-Year Students (Presentation-II)

Objective: Retrieve all students in their first academic year.

```
SELECT Name, Department, Year
FROM STUDENT
WHERE Year = 1;
```

Relational algebra: PROJECT[Name, Department, Year](SELECT[Year = 1](STUDENT))

Output: 14 rows: Aarav Sharma, Diya Patel, Ishan Verma, Ananya Reddy, Kabir Nair, Meera Joshi, Siddharth Rao, Tanvi Kulkarni, Aditya Sen, Pooja Hegde, Rohan Das, Sneha Pillai, Vikram Bhat, Neha Menon. Execution time 2.1 ms.

Query 2: Instructor-Course Inner Join (Presentation-II)

Objective: Correlate courses with their assigned instructors.

```
SELECT i.Name AS Instructor_Name, c.Course_Name, c.Category
FROM INSTRUCTOR i
JOIN COURSE c ON i.Instructor_ID = c.Instructor_ID;
```

Relational algebra: PROJECT[i.Name, c.Course_Name, c.Category](INSTRUCTOR JOIN[i.Instructor_ID = c.Instructor_ID] COURSE)

Instructor_Name	Course_Name	Category
Dr. Alan Turing	Intro to Python	Programming
Dr. Alan Turing	Database Systems	Database
Dr. Ada Lovelace	Cloud & Web Architecture	Cloud Computing
Dr. CV Raman	Circuits & Systems	Hardware

This confirms Dr. Alan Turing is assigned exactly 2 courses.

Query 3: Student Course Registrations (3-way join)

Objective: List enrollments with department and course name.

```
SELECT s.Student_ID, s.Name AS Student_Name, s.Department,
       c.Course_Name, e.Enrollment_Date
FROM ENROLLMENT e
JOIN STUDENT s ON e.Student_ID = s.Student_ID
JOIN COURSE c ON e.Course_ID = c.Course_ID
ORDER BY s.Student_ID;
```

Relational algebra: PROJECT[s.ID, s.Name, s.Dept, c.Course, e.Date]((ENROLLMENT JOIN STUDENT) JOIN COURSE)

Output: 30 enrollment records.

Query 4: Detailed Learning Progress (4-way join)

Objective: Track student progress through syllabus modules.

```
SELECT s.Name AS Student_Name, c.Course_Name, m.Module_Name,
       p.Status, p.Completion_Percentage, p.Last_Accessed
```

```
FROM PROGRESS p
JOIN STUDENT s ON p.Student_ID = s.Student_ID
JOIN MODULE m  ON p.Module_ID  = m.Module_ID
JOIN COURSE c  ON m.Course_ID  = c.Course_ID
ORDER BY s.Student_ID, m.Sequence_Order;
```

Output: 60 progress records with valid statuses (Completed at 100%, In Progress at 40-75%, Not Started at 0%).

Query 5: Assessment Performance and Score Percentage (4-way join)

Objective: Compute each score as a percentage of maximum marks.

```
SELECT s.Name AS Student_Name, m.Module_Name, a.Title AS Assessment_Title,
       a.Type AS Assessment_Type, r.Score, a.Max_Marks,
       ROUND((r.Score / a.Max_Marks) * 100, 2) AS Percentage
FROM RESULT r
JOIN STUDENT s    ON r.Student_ID    = s.Student_ID
JOIN ASSESSMENT a ON r.Assessment_ID = a.Assessment_ID
JOIN MODULE m     ON a.Module_ID     = m.Module_ID
ORDER BY Percentage DESC;
```

Output: 60 graded submissions with computed percentages.

Query 6: Department-wise Student Distribution (GROUP BY)

Objective: Aggregate student counts per department.

```
SELECT Department, COUNT(*) AS Total_Students
FROM STUDENT
GROUP BY Department
ORDER BY Total_Students DESC;
```

Relational algebra: GROUP[Department; COUNT(*) AS Total_Students](STUDENT)

Department	Total_Students	
Computer Science	13	
Information Technology	10	
Electronics	7	

Query 7: Course Completion Analytics (LEFT JOIN + aggregation)

Objective: Compute course-level average completion using an outer join.

```
SELECT c.Course_ID, c.Course_Name, c.Category,
       COUNT(DISTINCT m.Module_ID) AS Total_Modules,
       COUNT(p.Progress_ID) AS Tracked_Progress_Records,
       COALESCE(ROUND(AVG(p.Completion_Percentage), 2), 0.00) AS Avg_Completion_Rate
FROM COURSE c
JOIN MODULE m ON c.Course_ID = m.Course_ID
LEFT JOIN PROGRESS p ON m.Module_ID = p.Module_ID
GROUP BY c.Course_ID, c.Course_Name, c.Category
ORDER BY Avg_Completion_Rate DESC;
```

Output: One row per course (4 courses) with module count, tracked records and average completion rate.

11. UI Design and Screenshots

The frontend is a responsive administrative dashboard built with React 18, Vite, TypeScript and TailwindCSS. Every screen reads from and writes to the live MySQL database through the REST API.

11.1 Analytics Dashboard

Live KPI counters computed by SQL, department distribution chart (Recharts), cohort breakdown and recent enrollment feed.

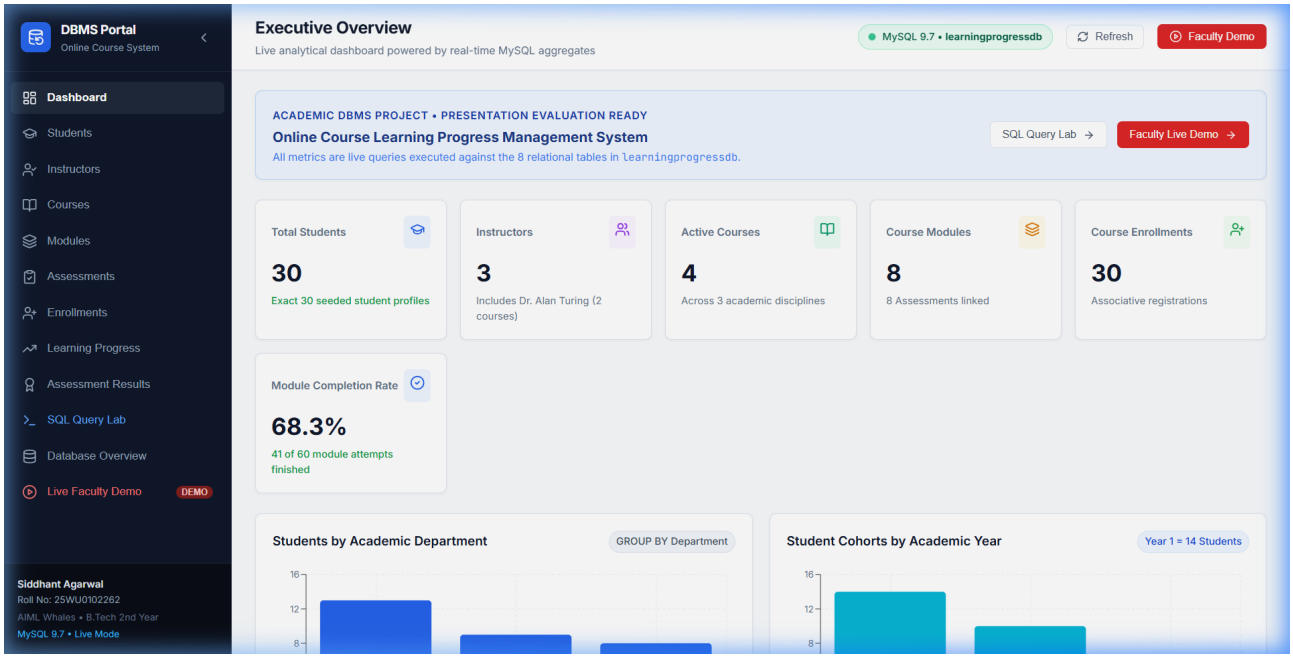


Figure 11.1: Dashboard analytics

11.2 Student Management Directory

Searchable and filterable directory with modal forms for insert and edit, and a relational details drawer.

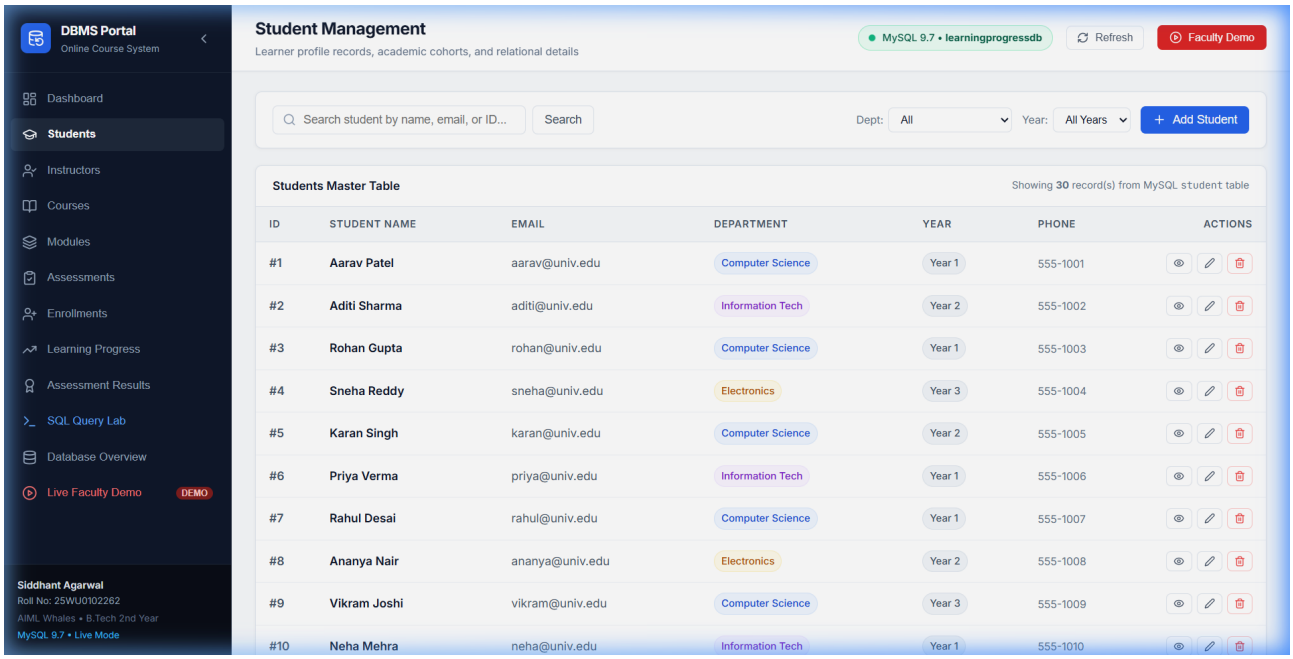


Figure 11.2: Students directory

11.3 Interactive SQL Query Lab

Runs the presentation queries with syntax display, execution timing and tabular results.

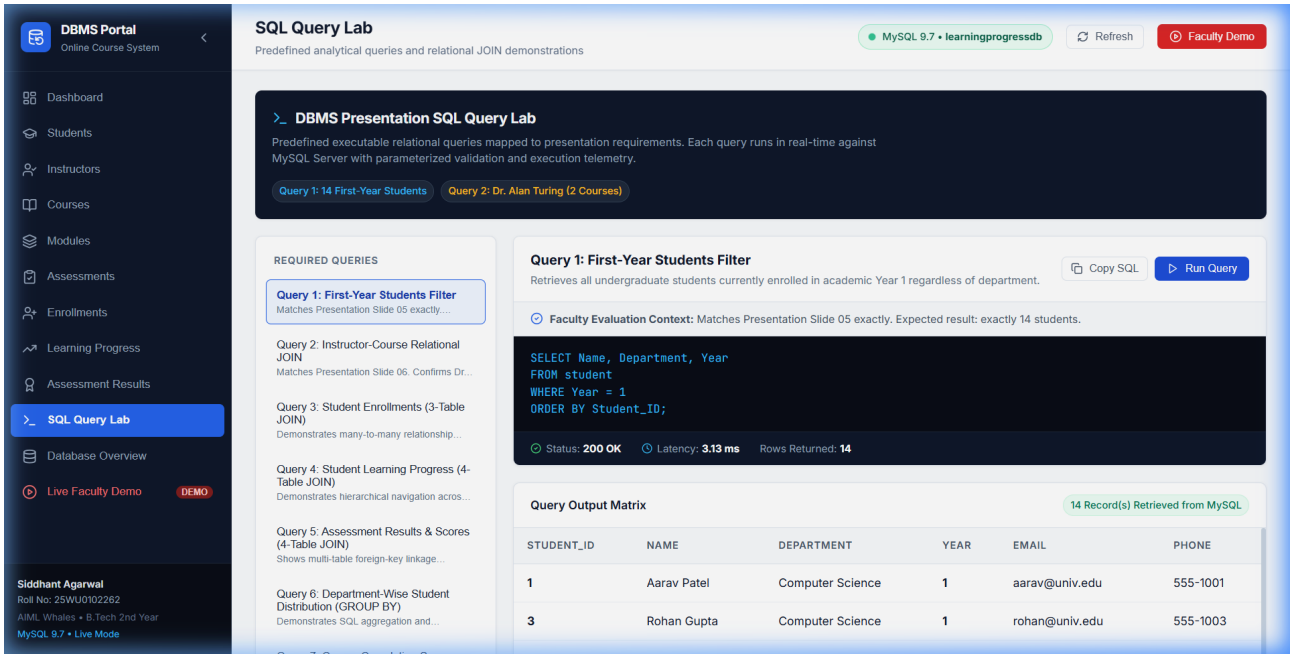


Figure 11.3: Query Lab, Query 1 (14 first-year students)

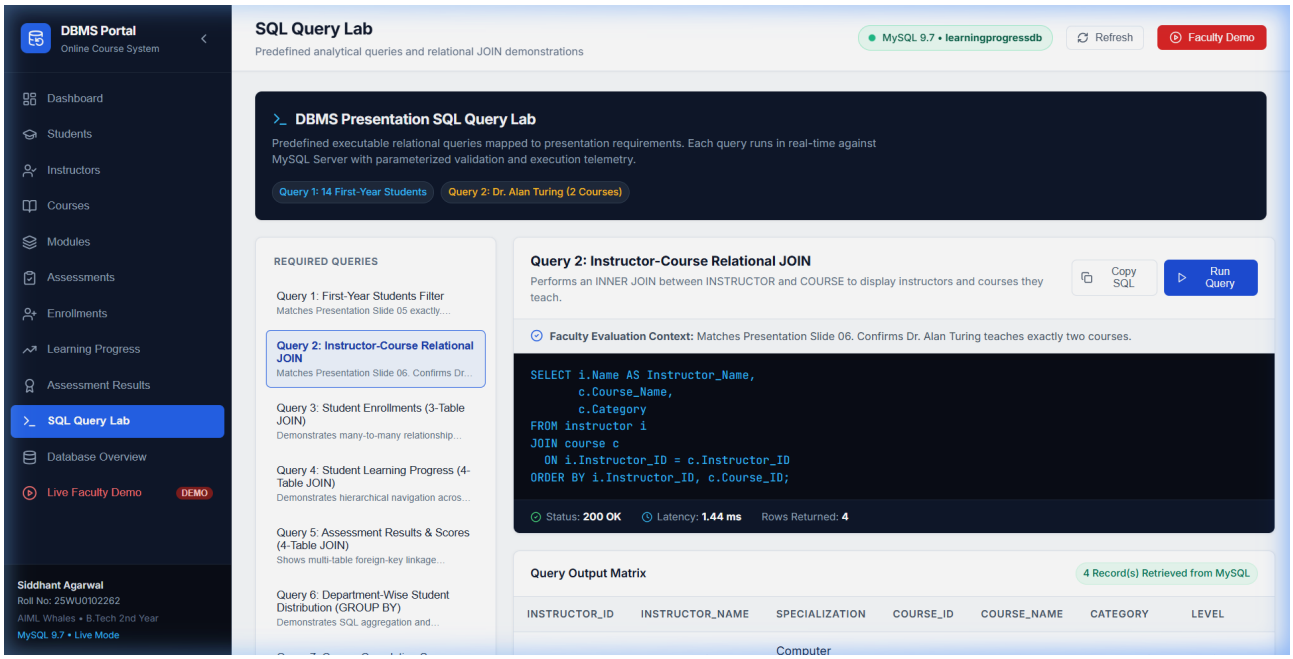


Figure 11.4: Query Lab, Query 2 (Dr. Alan Turing teaches 2 courses)

11.4 Database Overview

Live table row counters, schema introspection from information_schema.COLUMNS and a relationship overview.

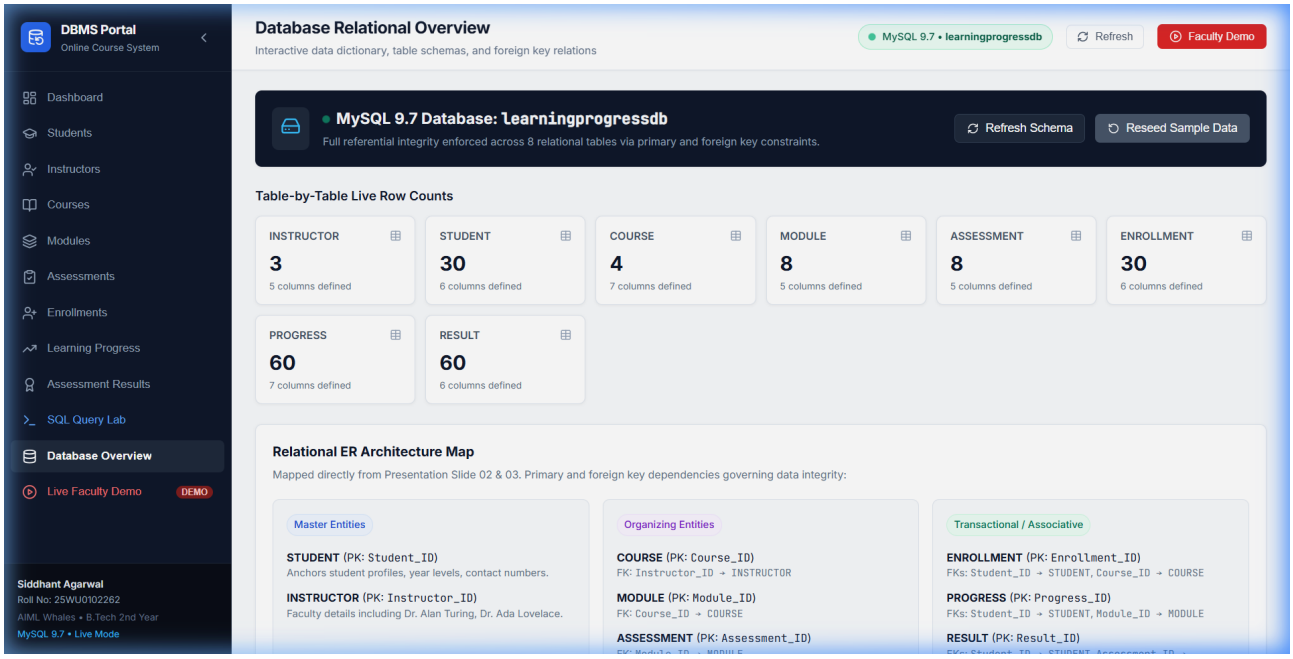


Figure 11.5: Database overview and schema

11.5 Faculty Evaluation Demonstration Flow

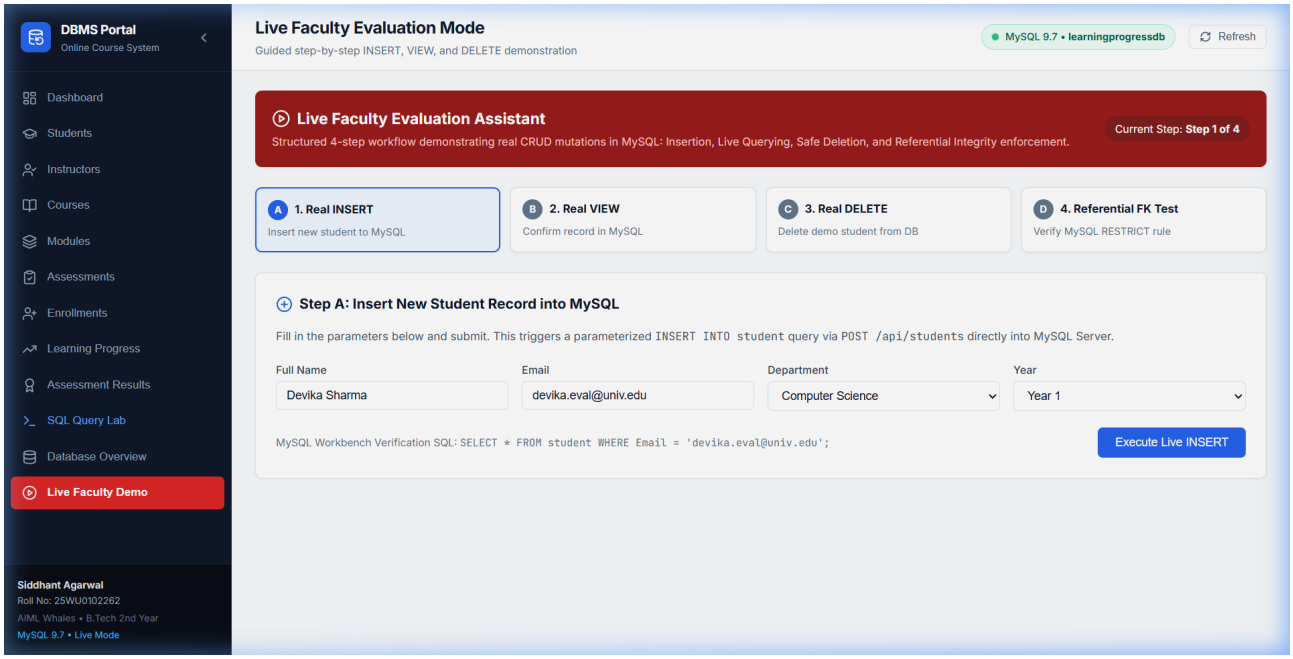


Figure 11.6: Step A - insert form for Student #31

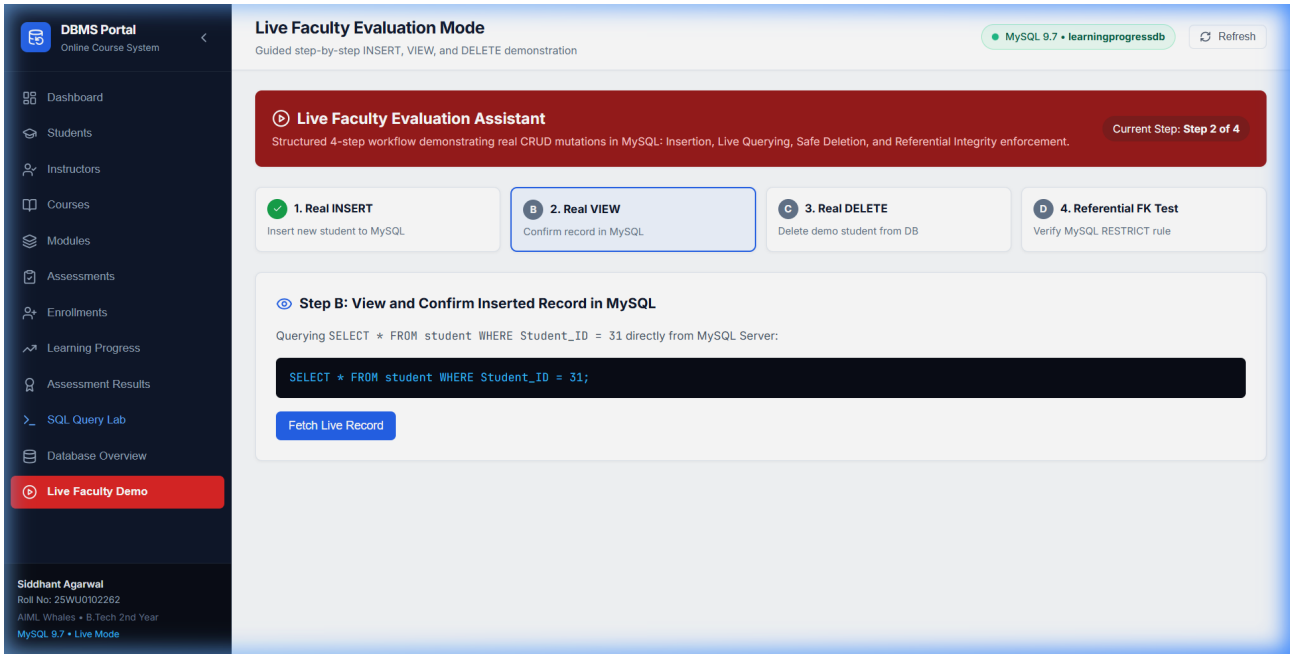


Figure 11.7: Step A - Student #31 inserted

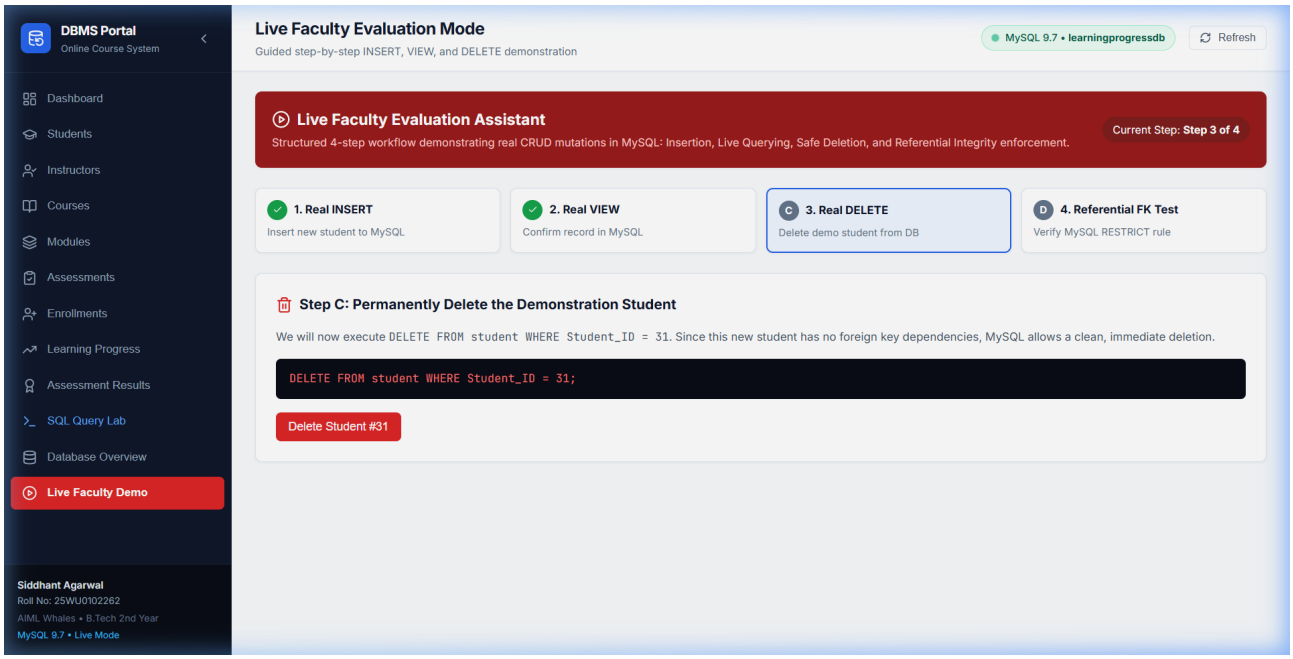


Figure 11.8: Step B - Student #31 queried from MySQL

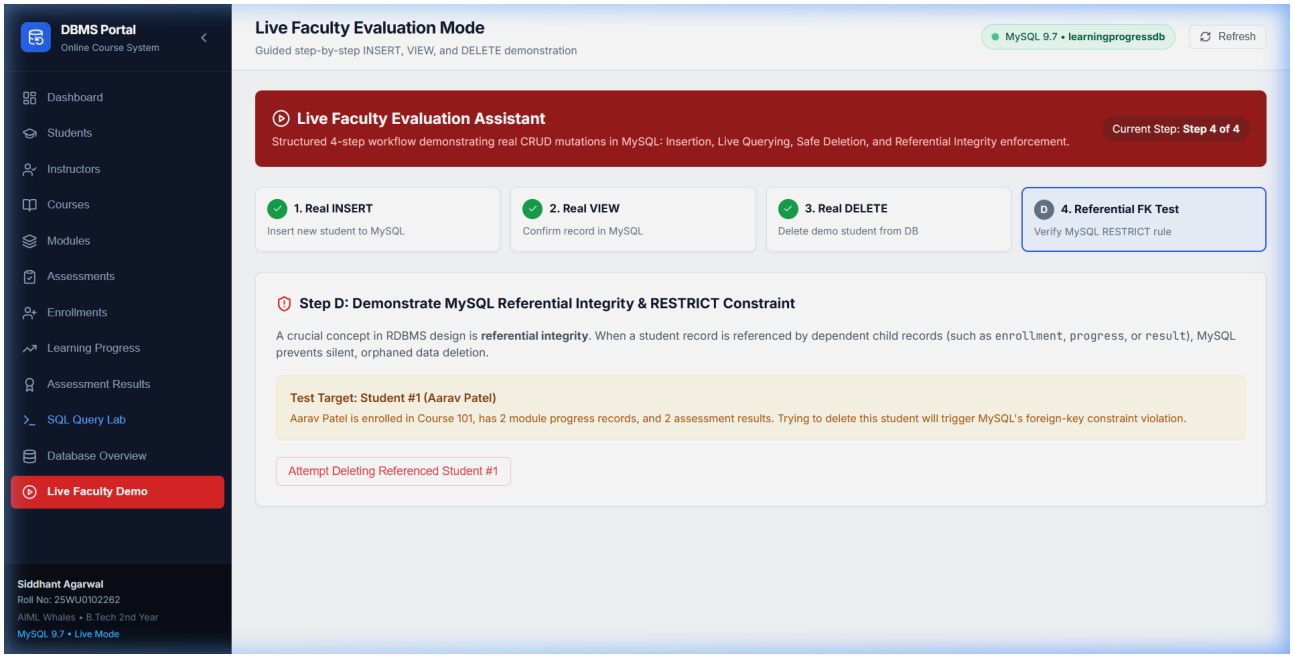


Figure 11.9: Step C - Student #31 deleted

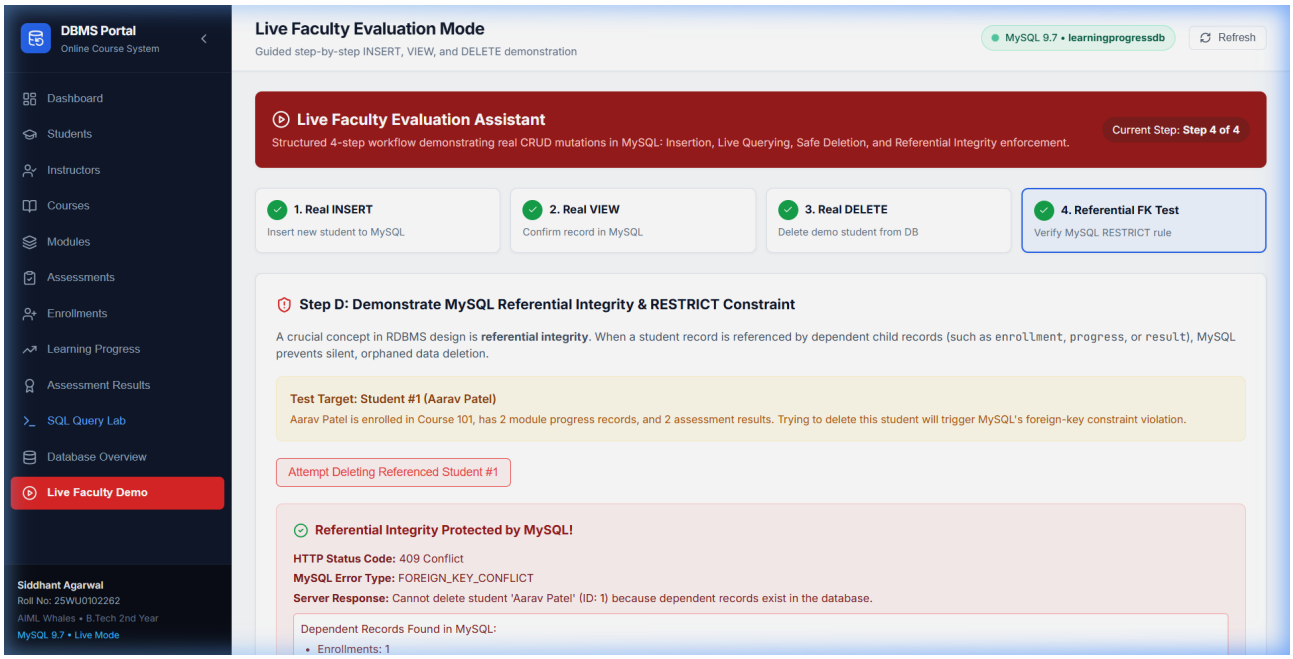


Figure 11.10: Step D - foreign key conflict when deleting Student #1

12. Implementation Details

12.1 Technology Stack

Tier	Technology
Presentation	React 18, Vite 6, TypeScript 5, TailwindCSS 3.4, React Router 6, Recharts 2, Lucide Icons (http://localhost:3000)
Application	Node.js v20, Express.js 4.x, mysql2/promise, dotenv, cors (http://localhost:5000)
Data	MySQL 9.7, InnoDB, database learningprogressdb

```
+-----------------------------------------------------+
|  PRESENTATION TIER   React + Vite + TypeScript      |
+-----------------------+-----------------------------+
                        | REST API (JSON over HTTP)
+-----------------------v-----------------------------+
|  APPLICATION TIER    Node.js + Express + error handler  |
+-----------------------+-----------------------------+
```

```
                        | mysql2/promise connection pool
+-----------------------v-------------------------------+
|  DATA TIER           MySQL 9.7 (InnoDB, ACID)          |
+--------------------------------------------------------+
```

12.2 Database Connectivity (backend/src/config/db.js)

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
    host: process.env.DB_HOST || 'localhost',
    port: parseInt(process.env.DB_PORT, 10) || 3306,
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME || 'learningprogressdb',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0,
    enableKeepAlive: true
});
```

12.3 Centralized Error Handling (backend/src/middleware/errorHandler.js)

Native MySQL error codes are translated into HTTP responses:

MySQL error	HTTP status	Message
ER_ROW_IS_REFERENCED_2 (1451)	409 Conflict	Cannot delete or modify record because it is referenced by dependent child records.
ER_DUP_ENTRY (1062)	409 Conflict	A record with this unique identifier already exists.
ER_NO_REFERENCED_ROW_2 (1452)	400 Bad Request	Referenced parent record does not exist.

12.4 Key Features

- Full CRUD for all 8 entities with dropdown selectors for foreign-key fields.
- Referential integrity protection with conflict dialogs on blocked deletes.
- SQL Query Lab executing the 7 benchmark queries with millisecond timing.
- Dashboard KPIs and schema introspection computed live from MySQL.

13. Testing (Test Cases and Results)

ID	Category	Action / SQL	Expected	Observed	Status
TC-01	DB startup	Connect via mysql2 pool	Connected to learningprogressdb	Logged: MySQL Database connected successfully	PASS
TC-02	Table count	SHOW TABLES;	8 tables	8 tables verified	PASS
TC-03	Seed check	SELECT COUNT(*) FROM STUDENT;	30	Returns 30	PASS
TC-04	Presentation Q1	COUNT of Year = 1 students	14	Returns 14	PASS
TC-05	Presentation Q2	Courses of Dr. Alan Turing	2	Returns 2 (Intro to Python, Database Systems)	PASS
TC-06	Orphan FK check	LEFT JOINs across child tables	0 orphans	0 orphan records	PASS
TC-07	CRUD insert	POST /api/students	201 Created	Student #31 created	PASS
TC-08	CRUD delete (unreferenced)	DELETE /api/students/31	200 OK	Student #31 deleted	PASS
TC-09	FK RESTRICT	DELETE /api/students/1	409 Conflict (1451)	Deletion blocked; modal shown	PASS
TC-10	CHECK constraint	Insert Completion_Percentage = 150	Rejected by MySQL	400 validation error returned	PASS

14. Conclusion and Future Enhancements

14.1 Conclusion

- Developed an 8-table relational schema normalized to BCNF.
- Verified referential integrity with RESTRICT foreign keys; no orphaned records.
- Seeded deterministic data (30 students, 14 in Year 1, Dr. Alan Turing teaching 2 courses) matching the Presentation-II benchmarks.
- Delivered a full-stack application with live CRUD, analytical queries and schema introspection against a real MySQL server.

14.2 Future Enhancements

- **Role-based access control:** separate Student, Instructor and Admin portals with JWT authentication.
- **Automated triggers:** stored procedures and triggers to update PROGRESS.Completion_Percentage when a RESULT is recorded.
- **Predictive analytics:** machine-learning models to flag at-risk students from module duration and score trends.

15. References

[1] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.

[2] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

[3] Oracle Corporation. *MySQL Reference Manual: InnoDB Storage Engine and Constraints*. dev.mysql.com.

[4] React Documentation. *React: The Library for Web and Native User Interfaces*. react.dev.

[5] Express.js Foundation. *Express - Fast, unopinionated, minimalist web framework for Node.js*. expressjs.com.

16. Appendix: GitHub Repository Link

Item	Detail
Repository	DBMS-Course-Project (public)
URL	https://github.com/asiddhant2405/DBMS-Course-Project

Repository structure

```
Presentation-I/    Problem statement, objectives, scope, conceptual ER
Presentation-II/   Schema, BCNF proofs, SQL scripts, presentation queries
Presentation-III/  UI source, MySQL connectivity, CRUD screenshots, demo flow
Project-Report/    Project report
database/          schema.sql, seed.sql, reset_and_seed.sql, verification.sql
backend/           Node.js + Express REST API
frontend/          React + Vite + TypeScript client
screenshots/       System verification screenshots
```

Environment configuration (.env)

```
PORT=5000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=learningprogressdb
NODE_ENV=development
```

Database initialization and running the application

```
mysql -u root -p -e "source database/schema.sql;"
mysql -u root -p -e "source database/seed.sql;"
mysql -u root -p -e "source database/verification.sql;"

# Terminal 1 - backend
cd backend && npm install && node src/server.js

# Terminal 2 - frontend
cd frontend && npm install && npm run dev -- --port 3000
```

Open `http://localhost:3000` in a web browser.